Sistema de Abastecimiento Konrad Gourmet

Documento de arquitectura: microservicios, arquitectura orientada a eventos y arquitectura hexagonal

Mauricio Jesús Iturriza Medina
Carlos Franco
Adrián David Corredor Camero
Isabela Castro

Arquitectura de Software, proyecto 1
Profesor Carlos Andrés López
Programa de Ingeniería de Sistemas
Fundación Universitaria Konrad Lorenz
Bogotá D. C., noviembre de 2026

1. Resumen de la decisión

Konrad Gourmet necesita que cuatro roles operativos (mesero, jefe de cocina, auxiliar de compras y director de compras) trabajen sobre los mismos datos desde todas las sucursales, con 200.000 usuarios concurrentes, 1.000 transacciones por segundo y una disponibilidad del 99,7 %. Ningún estilo arquitectónico cumple solo esas restricciones, así que la propuesta combina tres, cada uno en el nivel donde resuelve un problema distinto.

Tabla 1

Las tres arquitecturas de la propuesta

Arquitectura	Dónde vive	Qué decide	Requerimientos que la exigen	Dónde se ve
Microservicios	Nivel 2 del C4 (contenedores)	Once servicios desplegables, cada uno con su base de datos, detrás de un API Gateway	200.000 usuarios, 1.000 TPS, 99,7 %, computación distribuida, crecimiento del 200 %	Mapa (cajas azules) y página de despliegue
Orientada a eventos	Entre contenedores	Comunicación asíncrona entre roles y procesos automáticos por medio de diez tópicos	Correo certificado, alarma del 25 %, auditoría de cada CRUD, tendencias, desacople ante caídas	Mapa (banda ámbar y flechas punteadas)
Hexagonal	Niveles 3 y 4 del C4, dentro de cada servicio	Dominio aislado de la base de datos, del SOAP, del SMTP y del PDF	Cambio fácil de motor de base de datos, integración SOAP, tecnología libre, mantenimiento	Mapa (hexágonos) y páginas 4a a 4e

Las tres se leen juntas en la página 1 del archivo .drawio (figura 1): las cajas azules son los microservicios, la banda ámbar es el broker de eventos y los hexágonos verdes son el interior de cada servicio. A su alrededor aparecen cinco estilos y mecanismos de apoyo que completan a los tres principales: cliente-servidor con REST entre la SPA y el API Gateway, SOA con SOAP en la frontera con los sistemas externos, OAuth 2.0 con JWT para la seguridad, un Service Registry para descubrir instancias y un balanceador de carga activo-activo.

El profesor insistió en que «un diseño no es bueno o malo en sí mismo, sino con respecto a las restricciones que se le imponen» (López, 2026b, sesión 10). Por eso cada decisión de este documento se justifica contra un requerimiento no funcional del enunciado.

2. Método de análisis

2.1 Fuentes

Se analizaron tres fuentes. La primera es el enunciado del proyecto (López, 2026a), con sus requerimientos funcionales y diecinueve requerimientos no funcionales agrupados en seguridad, desempeño, interfaz gráfica, almacenamiento, mantenimiento, tecnología e integración. La segunda son las 23 ventanas del prototipo junto con el flujo web que las conecta botón por botón. La tercera son las sesiones de clase, en las que los grupos expusieron los estilos de arquitectura y el profesor presentó el modelo C4 (Brown, 2018) y los patrones de diseño (Gamma et al., 1994).

Tabla 2

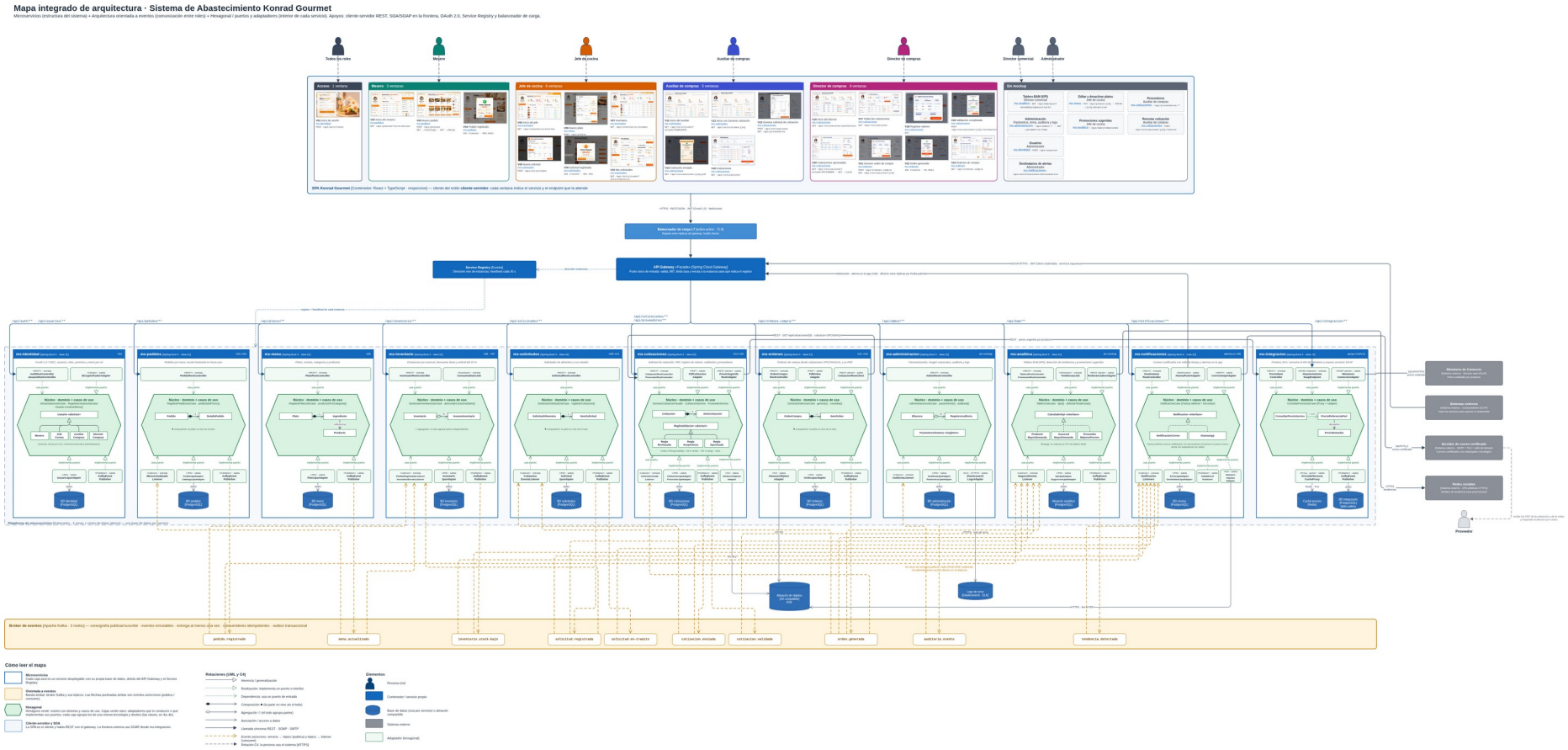
Dónde está cada nivel del C4

Nivel	Página del .drawio	Figura	Sección
1. Contexto	2	11	14
2. Contenedores	3a	12	14
3. Componentes	3b	13	14
4. Código	4a a 4e	8 a 10 y A1 a A5	12 y anexo A
Dinámico	6	7	10
Despliegue	5	14	14

Nota. Los cuatro niveles del C4 son obligatorios; el diagrama dinámico y el de despliegue son complementarios.

Figura 1

Mapa integrado de las tres arquitecturas



Nota. Página 1 del archivo Arquitectura_Konrad_Gourmet.drawio. De arriba abajo: personas por rol, SPA con las 23 ventanas, balanceador, API Gateway y Service Registry, los once microservicios con su núcleo hexagonal, el broker de eventos y, a la derecha, los sistemas externos.

19. Referencias

Brown, S. (2018). *The C4 model for visualising software architecture*. https://c4model.com

Gamma, E., Helm, R., Johnson, R. y Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

López, C. A. (2026a). *Proyecto #1: Implementación sistema de abastecimiento* [Enunciado del proyecto]. Arquitectura de Software, Fundación Universitaria Konrad Lorenz.

López, C. A. (2026b). *Sesiones de clase de Arquitectura de Software, sesiones 2 a 10* [Transcripciones]. Fundación Universitaria Konrad Lorenz.